Components, SCCs, and Degree Distributions of a Graph

Alex Liu
University of California, Irvine
aliu40@uci.edu

Code: https://github.com/training4usaco/HayesGraphChallenge

November 2026

1 Introduction

In this project, I present my process and methods for finding different attributes of a given undirected (and directed) graph, as well as a short analysis on time complexity and the results. Specifically, for each graph, we find:

1. **Connected components (CC):** the number of components when the edges are treated as *undirected*,
2. **Strongly connected components (SCC):** the number of components when the edges are treated as *directed*,
3. **Degree distributions:** the undirected degree, indegree, and outdegree of every node, plotted as histograms.

While there are multiple ways to count SCCs and CCs in a graph, the method that was ultimately settled on was to use Disjoint Set Union (DSU) with path compression to efficiently count connected components in the undirected graph, and use Kosaraju's algorithm to count the strongly connected components in the directed graph, as implemented in `sol.cpp`. In addition, as a form of verification, Tarjan's algorithm and a standard Depth First Search (DFS) were additionally implemented to count SCCs and CCs, respectively. This was implemented in `alternate.cpp` (See 3.3).

2 Input and Preprocessing

Since the number of edges is not given directly, we simply use a `while` loop to read edges until the end of the file. We build two `std::set` of pairs, one containing ordered pairs and the other containing unordered pairs. The two sets represent directed and undirected edges, which are then used to build out two adjacency lists, one for the forward graph and one for the reverse graph.

The reason we also need the reverse graph is for the second half of Kosaraju's algorithm. This will be discussed more in detail later (Section 3.2).

2.1 Duplicate Edges

But why create two sets in the first place? The reason comes from the fact that the inputs contain duplicate edges. While duplicates don't affect the SCC or CC counts (a duplicate edge will not change whether or not a path between two nodes exist), they affect degree counts. For example, if $u \to v$ appears twice, then both $\text{out}(u)$ and $\text{in}(v)$ end up one higher than they should be. Therefore, we'll compute two distributions for the node degrees, indegrees, and outdegrees distribution: one accounting for duplicate edges and one excluding duplicate edges.

Thus, we create two `std::sets`:

- `edges` is an `std::set` of ordered pairs (u, v). We use it to build `adj` and `radj`, and to compute the indegrees and outdegrees. This is used to represent the *directed* edges.
- `uedges` is a `std::set` of pairs stored as $(\min(u, v), \max(u, v))$, so that $u \to v$ and $v \to u$ is only counted once. This is used to represent the *undirected* edges.

Graph	Total Edges	Distinct directed	Distinct undirected
`n10`	100	60	39
`n100`	99	99	99
`n1000`	1999	1995	1994
`n10000`	32999	32992	32983
`s1`	1202	1202	1202

Table 1: A table of edge counts for before and after removing duplicates for both directed and undirected edges. Note: none of the graphs contained any self loops (edges pointing from a node to itself).

3 Methods

3.1 Connected Components

For connected components, we use a disjoint-set union (DSU) with path compression. The idea is that we keep track of disjoint sets that represent components. Adding an edge between two components is the equivalent of merging two sets. Specifically, every node starts in its own set, with `par[i] = i` and `sz[i] = 1`.

Then, for each edge (u, v), we call `merge(u, v)`, which finds the root of each endpoint and, if the roots are different, joins the two sets. Note that this essentially creates a tree-like structure within each set, with the root lookup having time complexity $O(h)$ where h is the height of the tree. To improve performance, we also use two standard optimizations:

1. **Small to large merging:** When merging, we always attach the root of the smaller set underneath the root of the larger one.

   This is useful because of the following observation: consider some node u. When its set gets merged into another set, its depth within its tree increases by exactly 1. Let's say $u \in S_a$ is being merged into set S_b to create S_m, where $|S_b| \geq |S_a|$. Then we know that

   $$|S_m| = |S_a| + |S_b| \geq 2 \cdot |S_a|.$$

   In other words, every time u is moved, the size of the set it belongs to *at least doubles*.

   Since a set can contain at most n nodes, the size can only double $\log_2 n$ times. Equivalently, the depth of any node within its set can increase to at most $\log_2 n$, so a single call to `find` takes at most $O(\log n)$ time.

   In total, we call `merge` once for each of the m edges, with each `merge` calling `find` twice. Therefore, the total time complexity is

   $$O(m \log n),$$

   where m is the number of edges and n is the number of nodes. But we can do better than this.

2. **Path compression:** Note that we only ever care about the *root* of each set, since that's what tells us which set a node belongs to. Therefore, we can rearrange the tree so that each lookup becomes much more efficient. Whenever `find(u)` recurses up the tree to find the root, we "path compress" and connect every node directly to the root of the tree. This way, finding the root of any node takes only a single step after compression.

With both of these optimizations, each operation takes amortized $O(\alpha(n))$ time, where α is the inverse Ackermann function.

This is basically constant. For reference, at $n = 10^{12}$, $\alpha(n) = 3$.

Once every edge has been processed, the number of connected components is just the number of nodes that are still their own parent. This works with both duplicate and non duplicate edges since merging two nodes that are in the same component is a no-op.

3.2 Strongly Connected Components

For strongly connected components, we use Kosaraju's algorithm.

Consider the graph where each SCC is a single node. We'll call this the *condensed graph*. This graph contains no cycles, since a cycle would mean that nodes within the two SCCs can reach each other, creating a larger SCC.

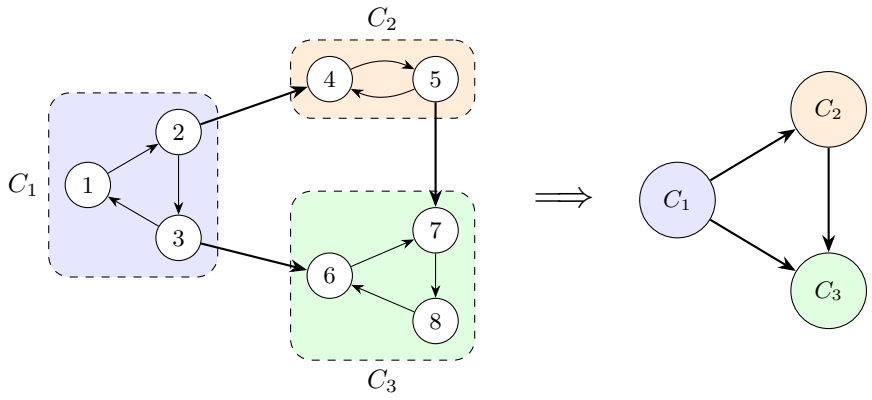


Figure 1: Left: a directed graph with its three SCCs outlined. Right: the corresponding condensation graph, where each SCC is shrunk into a single node. Note that the condensation graph has no cycles.

Since there are no cycles, there exists a node in the condensed graph that has no outgoing edges. By definition of an SCC, any DFS from a node within that SCC can reach the entire SCC, but can't access any other nodes. Thus, if we can find this "sink" node, we can run a DFS, mark those nodes as visited, remove it from the condensed graph, and repeat to find SCCs one at a time.

But how do we find this "sink" node? Well as it turns out, we can reverse the graph. Note that the condensed graph of the reverse graph is the same except the edges are also reversed (if a component is an SCC in the original graph, reversing the edge direction doesn't make another node reachable or unreachable, so it remains an SCC). Thus, the problem is now finding a "source" node (a node with no incoming edges).

To find a source, we run a DFS on the original graph and record the time we enter a node and the time we leave the node. Call these values $\text{tin}[u]$ and $\text{tout}[u]$. This is the equivalent of an Euler tour of the DFS tree.

For an SCC C, let

$$\text{tout}[C] = \max_{u \in C} \text{tout}[u],$$

the time the last node in C finishes.

Lemma 1. *If there is an edge from C_a to C_b in the condensed graph, then $\text{tout}[C_a] > \text{tout}[C_b]$.*

Proof. We consider two cases, depending on which SCC the DFS enters first.

- **C_a is entered first:** Let u be the first node we reach in C_a. Because there is an edge from C_a to C_b, every node in C_b is reachable from node u. This means that the DFS will visit and leave every node in C_b, and then return to the first node in C_a, or node u, before it leaves C_a (and C_b). Therefore, u finishes last, and $\text{tout}[C_a] > \text{tout}[C_b]$.

- C_b **is entered first:** This means that every node in C_b will be visited, and no nodes in C_a will be reached. The DFS will then eventually enter C_a, but this will be after the DFS has visited and left every node in C_b. Hence, again, $\text{tout}[C_a] > \text{tout}[C_b]$.

□

This means that the component with the largest tout will have no incoming edges. This is exactly what we're looking for! We can do a DFS and sort the nodes in decreasing order of tout (or equivalently, append each node to a list as soon as the DFS finishes with a node, and then reverse the list).

In summary, we run a DFS to order the nodes by decreasing tout, then go through the nodes in this order, and each time we reach an unvisited node, a DFS from it on the reversed graph finds exactly one SCC. We repeat this until every node has been visited, thus counting all SCCs.

Since this is essentially just two DFS passes, our total complexity comes out to

$$O(n+m),$$

where n is the number of nodes, and m is the number of edges.

Since we use a set to remove duplicates, we end with a total complexity of

$$O(n+m+m\alpha(n)+m\log m) = O(n+m\log m).$$

3.3 Verification

To check the results, `alternate.cpp` was also implemented. It uses a standards DFS to count components instead of DSU, and uses Tarjan's algorithm instead of Kosaraju's. The time complexity for these algorithms are the same as the ones presented earlier. Since these implementations serve only as a check and are not the main focus of this project, we will not go into detail about either algorithm. For all five graphs, both programs produce the same component counts and degree sequences.

$$\sum_u \text{in}(u) = \sum_u \text{out}(u) = m. \tag{1}$$

In other words, the sum of the indegrees matched with the sum of the outdegrees, which also matched with the number of edges (this applies to both with and without duplicate edges). Related to this, we also verified that

$$\sum_u \text{in}(u) + \sum_u \text{out}(u) \geq \sum_u \deg(u).$$

In fact, whenever duplicate edges are included, the two halves are equal (following from (1) and the fact that $\sum_u \deg(u) = 2m$ when duplicate edges are removed).

Another rather trivial thing that was verified was the fact that the number of SCCs was greater than or equal to the number of CCs. Consider the set of SCCs S, and the set of CCs C. We know that

$$\forall s \in S \; \exists\, c \in C \text{ such that } s \subseteq c.$$

Basically, every SCC is contained within a CC (since every node in a SCC can certainly reach other if the edges are treated as undirected) but a CC may contain multiple SCCs, so we know $|S| \geq |C|$.

4 Results

4.1 Component Counts

Table 2 shows the number of connected and strongly connected components for each graph.

Graph	Nodes	CCs	SCCs
`n10`	10	1	1
`n100`	100	12	100
`n1000`	1000	17	383
`n10000`	10000	12	814
`s1`	573	6	573

Table 2: Number of connected components (edges undirected) and strongly connected components (edges directed) for each graph.

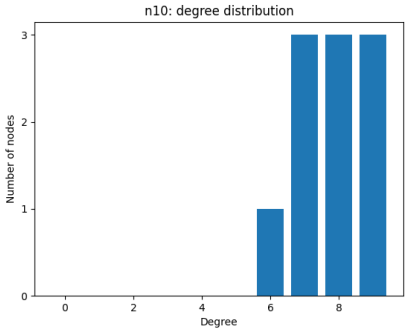
4.2 Degree Distributions

Figures 2–6 show the undirected degree, in-degree, and out-degree distributions for each graph. All of these are computed with duplicate edges removed.

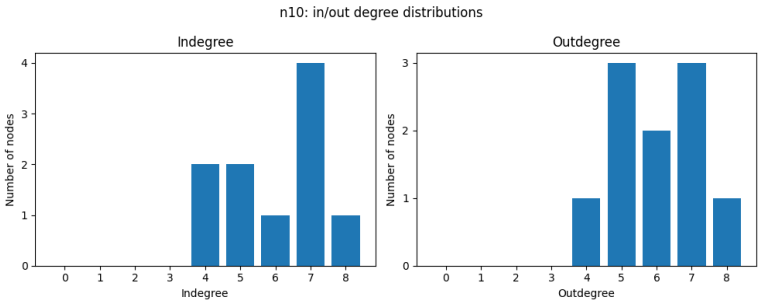
The degree distributions of graphs `n100`, `n1000`, and `n10000` are all slightly skewed right, but mostly bell shaped. Their indegree and outdegree distributions have a similar shape, except the right skew is more noticeable in the directed cases.

`n10` only has ten nodes, so its distribution doesn't take on a meaningful shape.

`s1` is strongly skewed right for all three types of degrees. Most nodes have a small degree, but a few nodes have much larger ones. Many nodes have no outgoing edges at all, and the most common indegree is two.

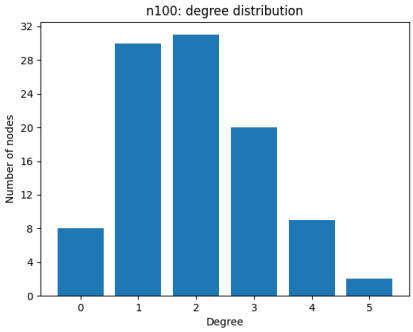


(a) Undirected degree.

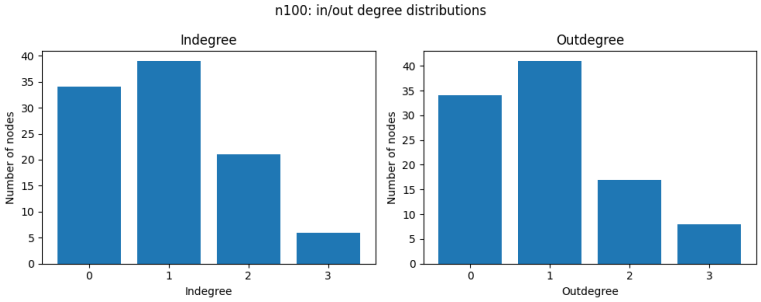


(b) In-degree and out-degree.

Figure 2: Degree distributions for `n10` (duplicates removed).

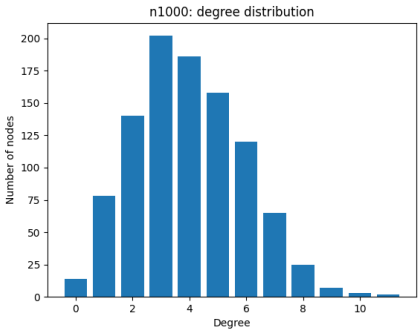


(a) Undirected degree.

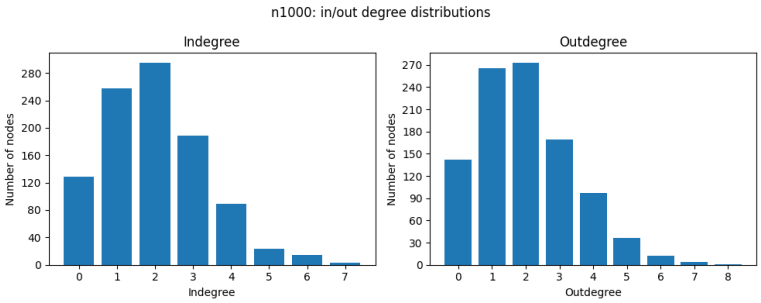


(b) In-degree and out-degree.

Figure 3: Degree distributions for `n100` (duplicates removed).

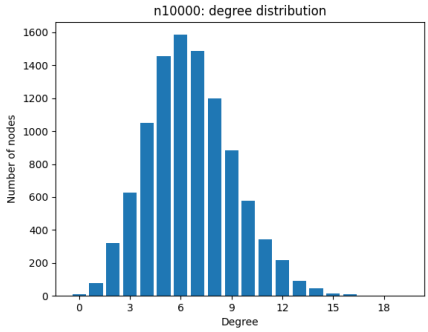


(a) Undirected degree.

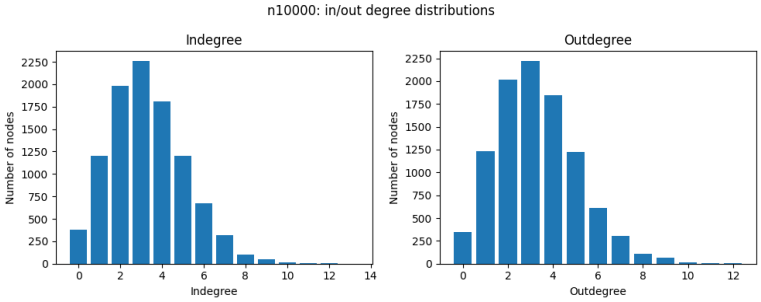


(b) In-degree and out-degree.

Figure 4: Degree distributions for `n1000` (duplicates removed).

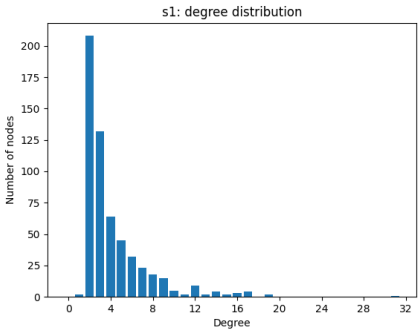


(a) Undirected degree.

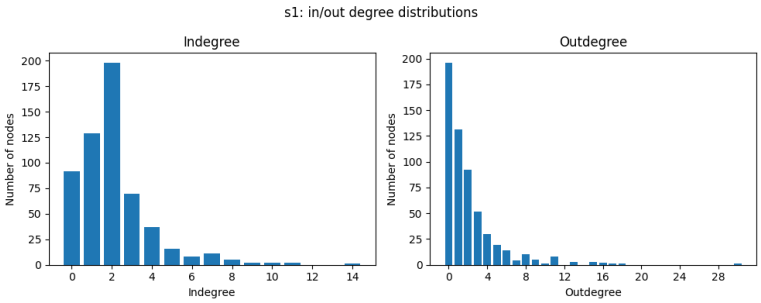


(b) In-degree and out-degree.

Figure 5: Degree distributions for `n10000` (duplicates removed).



(a) Undirected degree.



(b) In-degree and out-degree.

Figure 6: Degree distributions for `s1` (duplicates removed).

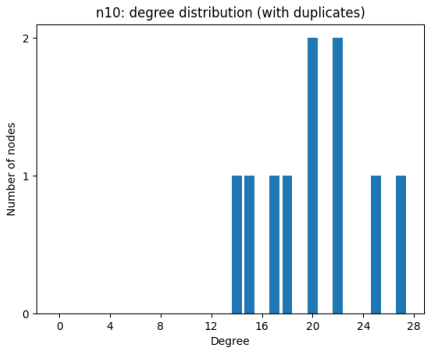
5 Discussion

5.1 Graph Structure

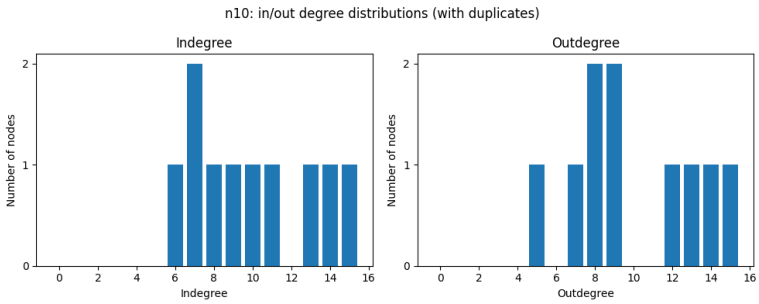
From the degree distributions of `n100`, `n1000`, and `10000` look like distributions coming from a randomly generated graph. If we fix the number of edges, and randomly pick the endpoints for each edge, it should be expected that for every node, its degree distribution should be close to the average, forming a general bell shaped curve. Since each endpoint is random, there is also no bias towards direction, so the indegree and the outdegree should similarly follow this bellshaped curve. `s1`, on the other hand, is quite different from the rest, as noted before. It seems like many nodes have much smaller degrees, with few nodes having much larger degrees. We can also note that it has as many SCCs as nodes (Table 2), meaning each node is its own SCC. In other words, the `s1` forms a directed acyclic graph (DAG), with a few nodes that fan out to many nodes. This type of graph actually resembles some sort of prerequsuite graph, or some sort of citation/dependency graph. This graph might've been generated by adding nodes one at a time, and randomly selecting a few nodes to create a directed edge. Perhaps there was also some bias towards nodes with higher degrees, creating these few nodes with large degree.

5.2 Effect of Duplicate Edges

Duplicate edges only make a significant difference for `n10`, where 100 input edges reduce to only 60 distinct directed edges (Table 1). Figure 7 shows its degree distributions with duplicates included. Compared to Figure 2, it seems the distribution was just simply shifted to the right.



(a) Undirected degree.



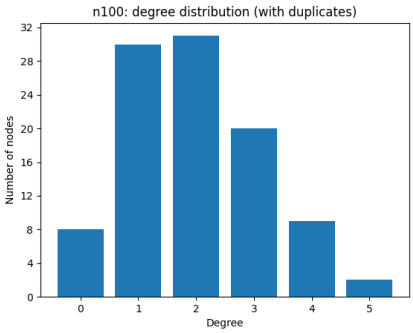
(b) In-degree and out-degree.

Figure 7: Degree distributions for `n10` with duplicate edges included.

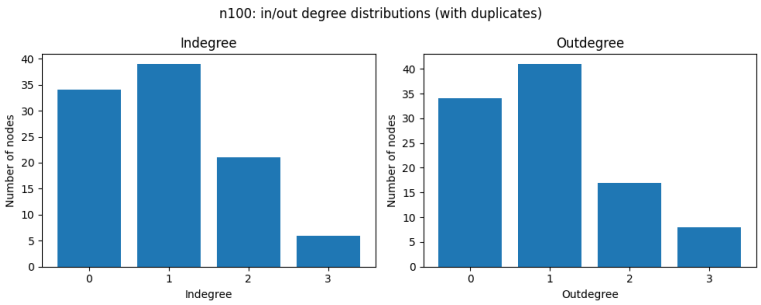
The other contains few or no duplicate edges, so their degree distributions are quite similar with or without duplicates. In fact, `n100` and `s1` contain no duplicates at all, so their degree distributions are exactly the same. For the rest of the graphs, we include their degree distribution with duplicate edges in the Appendix A.

A Distributions With Duplicate Edges

This appendix contains the undirected degree, in-degree, and out-degree distributions computed *with* duplicate edges for the graphs where duplicates make little or no difference. The `n100` and `s1` plots are identical to their duplicates-removed versions, and the `n1000` and `n10000` plots differ by only a handful of edges. The with-duplicates plots for `n10` appear in Figure 7.

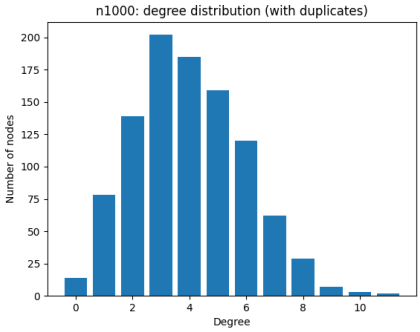


(a) Undirected degree.

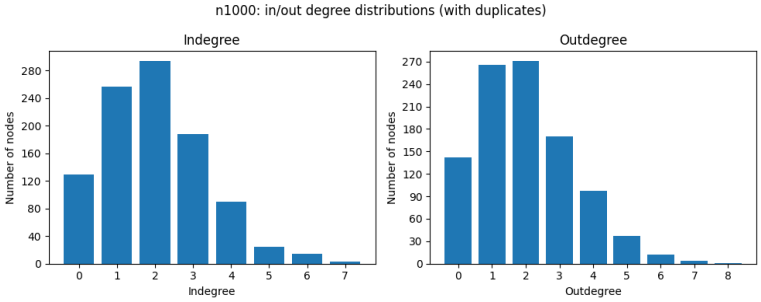


(b) In-degree and out-degree.

Figure 8: Degree distributions for `n100` (with duplicates).

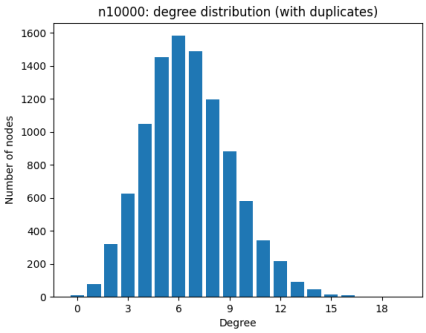


(a) Undirected degree.

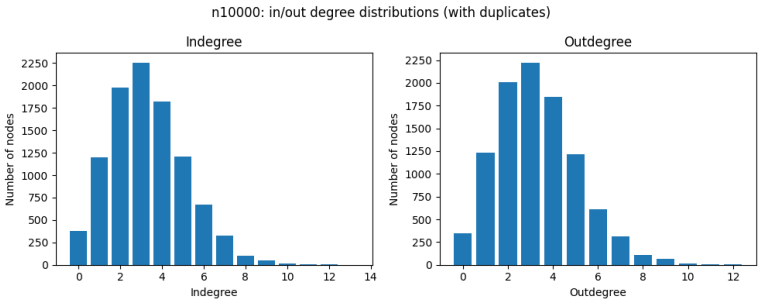


(b) In-degree and out-degree.

Figure 9: Degree distributions for `n1000` (with duplicates).

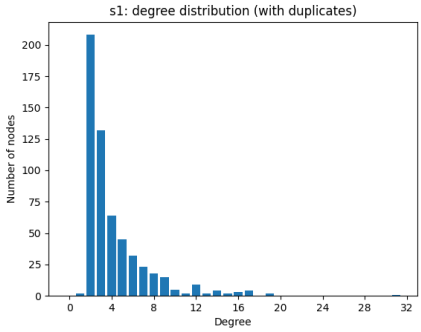


(a) Undirected degree.

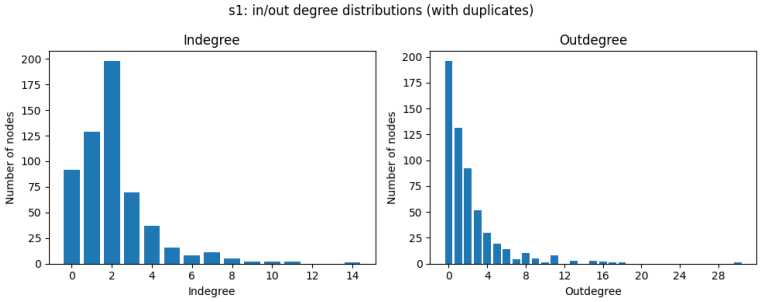


(b) In-degree and out-degree.

Figure 10: Degree distributions for `n10000` (with duplicates).



(a) Undirected degree.



(b) In-degree and out-degree.

Figure 11: Degree distributions for `s1` (with duplicates).